Runtime Environment Design for Function Calls with Activation Records + Heap Management and Garbage Collection : A Case Study

Case study Report submitted in partial fulfilment of the requirements for the award of the Degree of B.E in Computer Science and Engineering

By

Jessica	160623733146
K.Vignitha	160623733150
Shaik Karishma	160623733177
T. Akshitha	160623733185
V. Lasya	160623733187
Pedaprolu S S L Katyayani	160624733315

Under the Guidance of

Assistant Professor, Department of Computer Science & Engineering

Department of Computer Science and Engineering

Stanley College of Engineering & Technology for Women(Autonomous)

Chapel Road, Abids, Hyderabad – 500001

(Affiliated to Osmania University, Hyderabad, Approved by AICTE, Accredited by NBA & NAAC with A Grade)

Stanley College of Engineering & Technology for Women

(Autonomous)

Chapel Road, Abids, Hyderabad – 500001

(Affiliated to Osmania University, Hyderabad, Approved by AICTE, Accredited by NBA & NAAC with A Grade)

CERTIFICATE

This is to certify that case study report entitled Runtime Environment Design for **Function Calls with Activation Records + Heap Management and Garbage Collection** is being submitted By

Jessica	160623733146
K.Vignitha	160623733150
Shaik Karishma	160623733177
T Akshitha	160623733185
V. Lasya	160623733187
Pedaprolu S S L Katyayani	160623733315

In partial fulfilment for the award of the Degree of Bachelor of Engineering in Computer Science & Engineering to the Osmania University, Hyderabad is a record of bonafide work carried out under my guidance and supervision. The results embodied in this Problem statement report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Guide

Mrs. Gnana Prasuna

Professor, Dept. of CSE

Head of the Department

Dr. R Madana Mohana

Professor & Head , Dept. of CSE

STANLEY COLLEGE OF ENGINEERING AND TECHNOLOGY FOR WOMEN
(Autonomous)
Chapel Road, Abids, Hyderabad.
Department of Computer Science & Engineering

Institute Vision:

Empowering girl students through professional integrated with values and character to make an Impact in the world

Institute Mission:

M1: *Enabling quality engineering education for girl students to make them competent and confident to succeed in professional practice and advanced learning.*
M2: *Providing state-of-art-facilities and resources towards world class education.*
M3: *Integrating qualities like humanity, social values, ethics, leadership towards their contribution to society.*

Department Vision:

Empowering girl students with the contemporary knowledge in Computer Science and Engineering for their success in life.

Department Mission:

M1: *To impart quality education for girl students to learn and practice various hardware and software platforms prevalent in industry.*
M2: *To achieve self-sustainability and overall development through Research and Development activities.*
M3: *To provide education for life by focusing on the inculcation of human & moral values through and honest and scientific approach*
M4: *To groom students with good attitude, team work and personality skills.*

Programme Educational Objectives

PEO1: Our graduates shall have enhanced skills and contemporary knowledge in software and hardware technologies for professional excellence, towards successful employment, advanced learning and research.

PEO2: Our graduates shall have life-long learning attitude, innovation and creativity to master latest technologies, devise solutions for realistic and social issues in the society.

PEO3: Our graduates have good attitude and personality skills, ethical values, teamwork and leadership skill towards professionalism and ethical practices within the organization and the society.

Program Outcomes:

PO1: Engineering Knowledge: Apply knowledge of mathematics and science, with fundamentals of Computer Science & Engineering to be able to solve complex engineering problems related to CSE.

PO2: Problem Analysis: Identify, Formulate, review research literature and analyze complex engineering problems related to CSE and reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.

PO3: Design/Development of solutions: Design solutions for complex engineering problems related to CSE and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety and the cultural societal and environmental considerations.

PO4: Conduct Investigations of Complex problems: Use research–based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5: Modern Tool Usage: Create, Select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modeling to computer science related complex engineering activities with an understanding of the limitations.

PO6: The Engineer and Society: Apply Reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the CSE professional engineering practice.

PO7: Environment and Sustainability: Understand the impact of the CSE professional engineering solutions in societal and environmental contexts and demonstrate the knowledge of, and need for sustainable development.

PO8: Ethics: Apply Ethical Principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9: Individual and Team Work: Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary Settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large such as able to comprehend and with write effective reports and design documentation, make effective presentations and give and receive clear instructions.

PO11: Problem statement Management and Finance: Demonstrate knowledge and understanding of the engineering management principles and apply these to one's own work, as a member and leader in a team, to manage Problem statements and in multi disciplinary environments.

PO12: Life-Long Learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning the broadest context of technological change.

Program Specific outcomes:

PSO1: Problem-Solving Skills: The ability to apply standard practices and strategies in software Problem statement development using open-ended programming environments to deliver a quality product for the benefit of students.

PSO2: Design, Implement, Test and Evaluate a computer system, component or algorithm to meet desired needs and to solve a computational problem.

ABSTRACT

The Runtime Environment Design System is developed to demonstrate how memory management and program execution take place during runtime. The case study focuses on three major components: activation records, heap memory allocation, and garbage collection. Activation records are used to simulate function calls and manage local variables through a stack-based approach, while heap memory is utilized for dynamic memory allocation during program execution. A simple garbage collection mechanism is implemented to identify and remove unused memory objects, improving memory utilization.

The system is designed with a GUI-based visualization approach to provide a clear understanding of stack and heap memory organization. It helps users observe function call behavior, memory allocation, and memory deallocation in an interactive manner. The project mainly aims at conceptual understanding rather than real operating system-level memory management.

This system is useful for students and beginners to understand compiler runtime environments, memory management techniques, and system-level programming concepts. It can further be extended with advanced garbage collection algorithms, optimized memory handling techniques, and enhanced visualization features for real-world applications.

Tables of Contents

LIST OF FIGURES

1. INTRODUCTION

A runtime environment is an essential part of a computer system that manages the execution of programs during runtime. It provides the necessary support for handling function calls, memory allocation, variable storage, and program execution flow. Efficient runtime memory management is important for improving program performance, reducing memory wastage, and ensuring reliable execution.

This project, “Runtime Environment Design,” focuses on simulating the major components involved in runtime memory management, namely activation records, heap memory allocation, and garbage collection. Activation records, also known as stack frames, are used to manage function calls and local variables using a stack structure. Heap memory allocation enables dynamic creation of objects during program execution, while garbage collection helps in automatically removing unused memory objects to improve memory efficiency.

The system is designed as a GUI-based application to visually represent stack and heap memory organization. Through graphical visualization, users can clearly understand how memory is allocated, used, and released during program execution. The project mainly aims to provide conceptual understanding rather than actual operating system-level memory management.

In Compiler Design, the runtime environment plays a significant role because a compiler not only translates source code into machine code but also organizes how the program will execute in memory. The compiler is responsible for generating activation records for function calls, managing variable storage, handling parameter passing mechanisms, and supporting dynamic memory allocation. Proper runtime environment design ensures efficient execution of compiled programs and helps in optimizing memory usage and execution speed.

Understanding runtime environments is essential in compiler construction because it connects the compile-time processes with actual program execution. Concepts such as stack management, heap allocation, and garbage collection are fundamental for implementing modern programming languages and virtual machines.

1.1 About Problem statement:

This case study focuses on designing the runtime environment of a program, which manages memory and execution during program run time. It includes handling function calls using activation records (stack frames), heap memory allocation, and garbage collection. The runtime environment is a crucial part of a compiler that ensures efficient memory usage and correct execution of programs.

1.2 Objectives of the Problem statement

- To understand runtime memory organization
- To simulate activation records (call stack)
- To implement heap memory allocation
- To demonstrate garbage collection techniques
- To design a GUI-based system for visualization
- To understand memory management in real systems

1.3 Scope of the Problem statement

- Simulates function calls using a stack
- Demonstrates heap allocation using dynamic objects
- Implements a simple garbage collection mechanism
- Limited to conceptual understanding (not real OS-level memory management)
- Can be extended to advanced GC algorithms and memory optimization

1.4 Advantages

- Helps in understanding runtime memory structure
- Visualizes stack and heap clearly using GUI
- Demonstrates function call behavior
- Improves knowledge of memory management
- Useful for learning system-level programming

1.5 Disadvantages

- Simplified simulation (not real memory addresses)
- Garbage collection is basic (not optimized)
- Does not include advanced techniques like reference counting or generational GC
- Limited scalability

1.6 Applications

- Compiler design and implementation
- Operating Systems
- Programming language runtimes (Java, Python, etc.)
- Memory management systems
- Debugging and performance analysis tools

1.7 Hardware & Software Requirements

Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4GB minimum
- Storage: 500MB free space

Software Requirements

- Java JDK 8 or above
- IDE (Eclipse / IntelliJ / VS Code)
- Operating System: Windows/Linux/Mac

2. IMPLEMENTATION

2.1 Code Snippets Implementation

1. Stack (Activation Record) Initialization

```
Stack callStack = new Stack<>();
```

2. Heap Memory Representation

```
DefaultListModel heap = new DefaultListModel<>();
```

3. Function Call (Push Operation)

```
push.addActionListener(e -> {
callStack.push("Func" + (callStack.size() + 1));
updateStack(stackArea); });
```

4. Function Return (Pop Operation)

```
pop.addActionListener(e -> {
if (!callStack.isEmpty())
callStack.pop();
updateStack(stackArea);
});
```

5. Heap Allocation

```
allocate.addActionListener(e -> {
heap.addElement("Object" + (heap.size() + 1));
 });
```

6. Garbage Collection

```
gc.addActionListener(e -> {
if (!heap.isEmpty())
heap.remove(0);
});
```

7. Stack Display Update

```
void updateStack(JTextArea area) {
area.setText("");
for (String s : callStack) {
area.append(s + "\n");
} }
```

2.1 Code Implementation

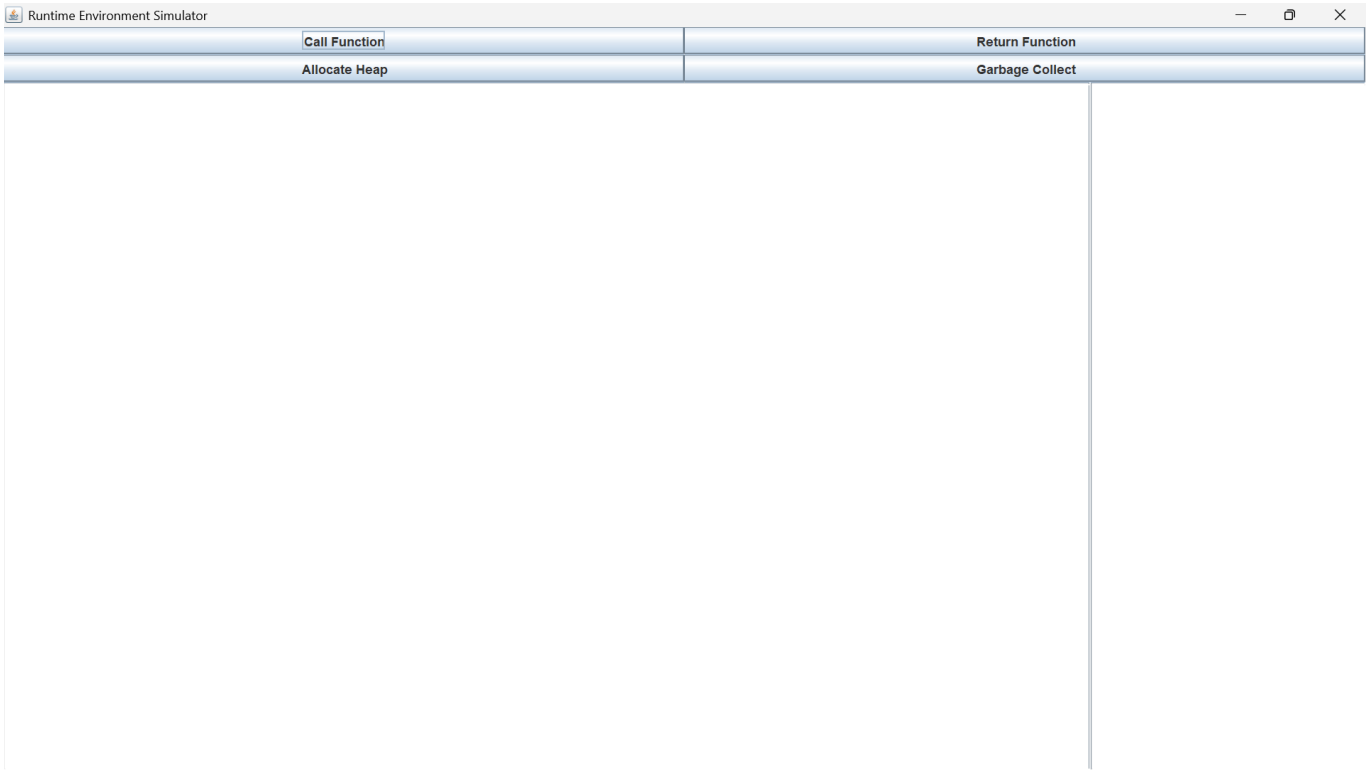
JAVA

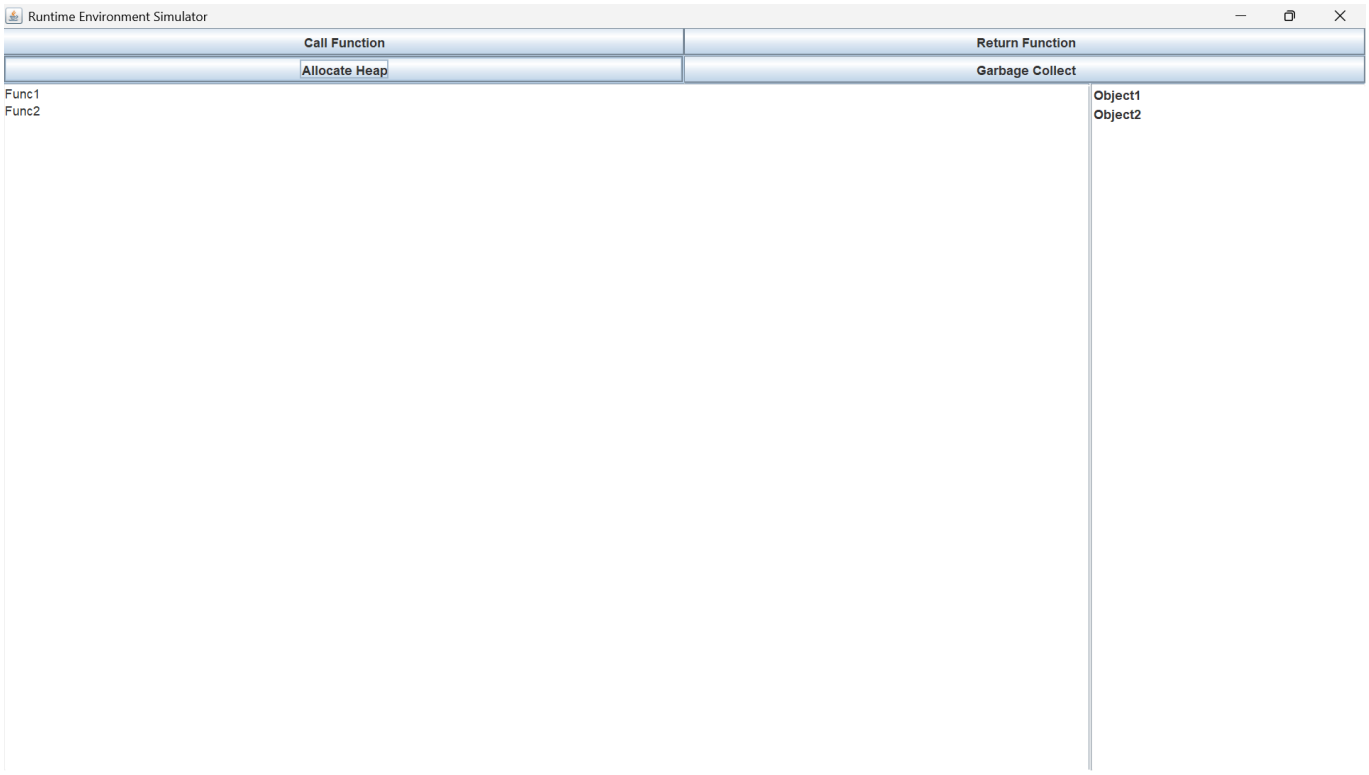
```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.Stack;
public class RuntimeGUI
{
static Stack callStack = new Stack<>();
static DefaultListModel heap = new DefaultListModel<>();
public static void main(String[] args)
{
JFrame frame = new JFrame("Runtime Environment Simulator");
frame.setSize(500, 400);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
JTextArea stackArea = new JTextArea();
JList heapList = new JList<>(heap);
JButton push = new JButton("Call Function");
JButton pop = new JButton("Return Function");
JButton allocate = new JButton("Allocate Heap");
JButton gc = new JButton("Garbage Collect");
push.addActionListener(e ->
{
callStack.push("Func" + (callStack.size() + 1));
updateStack(stackArea);
}
);
pop.addActionListener(e ->
{
if (!callStack.isEmpty()) callStack.pop();
```

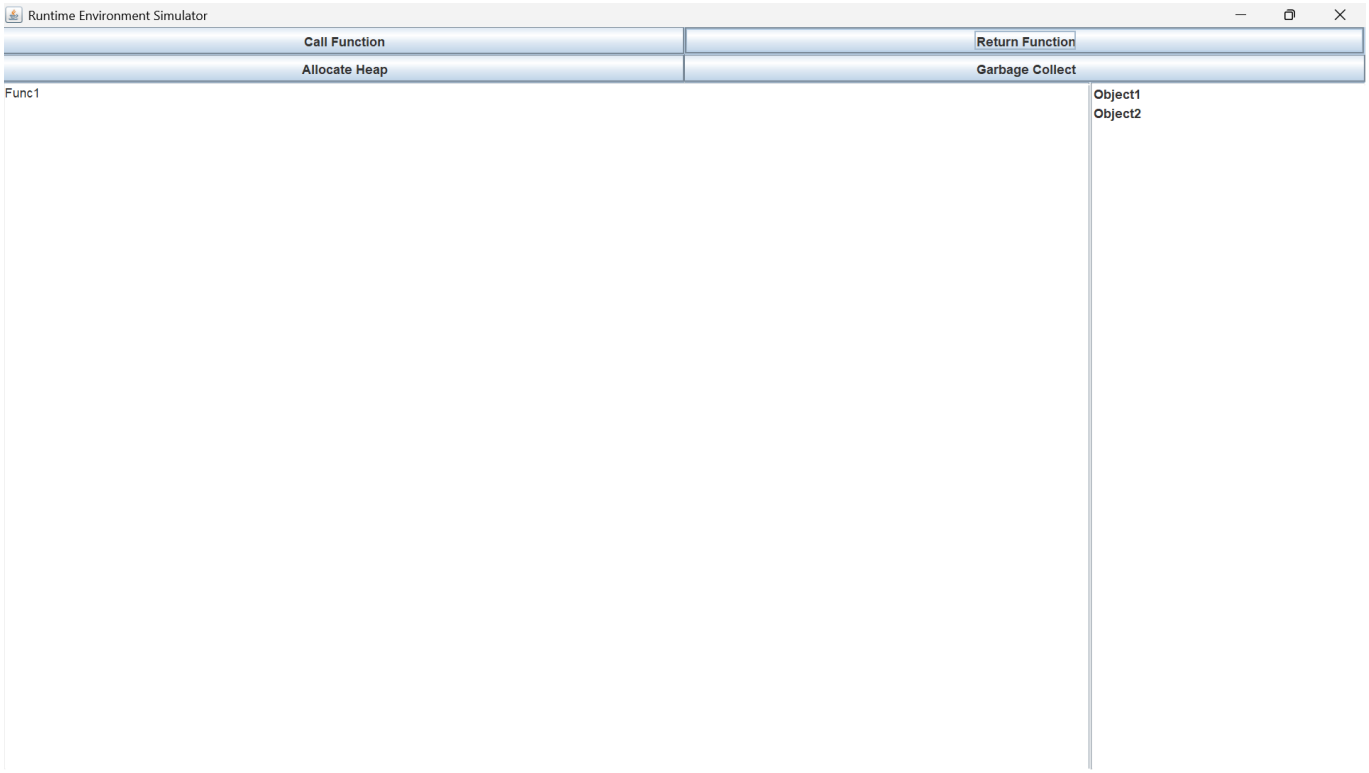
```
updateStack(stackArea);
}
);
allocate.addActionListener(e ->
{
heap.addElement("Object" + (heap.size() + 1));
}
);
gc.addActionListener(e ->
{
if (!heap.isEmpty())
heap.remove(0);
// simple GC
 }
);
 JPanel panel = new JPanel(new GridLayout(2,2));
panel.add(push);
panel.add(pop);
panel.add(allocate);
panel.add(gc);
frame.setLayout(new BorderLayout());
frame.add(panel, BorderLayout.NORTH);
frame.add(new JScrollPane(stackArea), BorderLayout.CENTER);
frame.add(new JScrollPane(heapList), BorderLayout.EAST);
frame.setVisible(true);
}
static void updateStack(JTextArea area)
{
area.setText("");
for (String s : callStack)
{
```

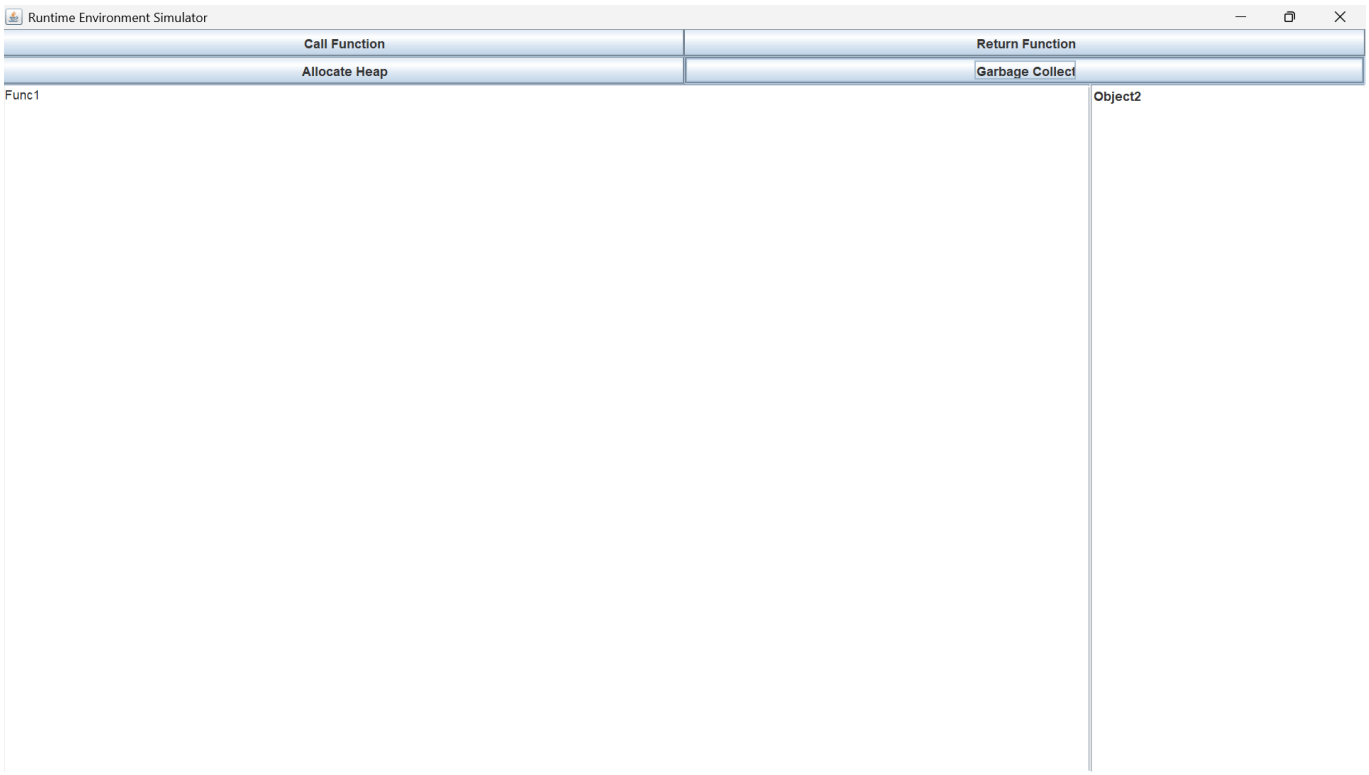
```
area.append(s + "\n");
}
}
}
```

Output:


Runtime Environment Simulator
Call Function
Return Function
Allocate Heap
Garbage Collect


Runtime Environment Simulator
Call Function
Return Function
Allocate Heap
Garbage Collect
Func1
Func2
Object1
Object2


Runtime Environment Simulator
Call Function
Return Function
Allocate Heap
Garbage Collect
Func1
Object1
Object2


Runtime Environment Simulator
Call Function
Return Function
Allocate Heap
Garbage Collect
Func1
Object2

3. RESULT ANALYSIS

The system successfully simulates runtime memory behavior using stack and heap.

Observations

- Function calls are stored in stack (LIFO order)
- Heap allocation dynamically adds objects
- Garbage collection removes unused objects
- GUI provides clear visualization of memory
- Demonstrates real-time memory changes

Example Execution

1) Call Function Twice

Stack:

Func1

Func2

2) Allocate Heap Objects

Heap:

Object1

Object2

3) Return Function

Stack:

Func1

4) Garbage Collection

Heap:

Object2

Conclusion

This case study demonstrates how runtime environments manage memory using stack and heap. It shows how function calls are handled using activation records and how garbage collection helps in efficient memory utilization. The project provides a strong foundation for understanding memory management in compilers and programming languages.

4. REFERENCES

[1] Aho, M. S. Lam, R. Sethi, and J. D. Ullman - *Compilers: Principles, Techniques, and Tools*

[2] Java Documentation - https://docs.oracle.com

[3] Operating System Concepts - Silberschatz

[4] GeeksforGeeks – Runtime Environment in Compiler Design

[5] TutorialsPoint – Memory Management